

Relación entre los Diagramas de Clases y la Programación Orientada a Objetos (POO)

Nombres: Zharick Sofia Rodriguez Gutierrez Y Mateo Moreno López

Ficha:3278645

Presentado a: Isaura Suarez

Índice

1. Introducción	pág. 3
2. Desarrollo	pág. 3
2.1 Conceptos clave en POO	pág. 3
2.2 El diagrama de clases	pág. 3
3. Relación entre Diagramas de Clases y la POO	pág. 4
4. Ejemplo	pág. 5
.....	pág. 6
5. Conclusión	pág. 6
6. Bibliografía	pág. 7

1. Introducción

La Programación Orientada a Objetos (POO) es un paradigma ampliamente utilizado que organiza el software mediante **objetos** con atributos y métodos. Antes de programar, es recomendable planificar la estructura del sistema; para ello se emplean los **diagramas de clases** (UML). Estos diagramas permiten visualizar las clases, sus atributos, métodos y relaciones, sirviendo como guía para la implementación en POO.

2. Desarrollo

2.1. Conceptos clave POO

- **Clase:** Plantilla que define atributos y métodos.
- **Objeto:** Instancia concreta de una clase.
- **Atributos:** Propiedades que describen un objeto (ej. nombre, edad).
- **Métodos:** Acciones que puede realizar el objeto (ej. saludar()).
- **Encapsulación:** Protección de datos internos mediante interfaces controladas.
- **Herencia:** Reutilización de atributos y métodos de una clase base.
- **Polimorfismo:** Un mismo nombre de método puede comportarse distinto según la clase.

2.2. El diagrama de clases

El diagrama de clases es una representación visual que muestra:

- Nombre de cada clase.
- Atributos y métodos de cada clase.
- Relaciones entre clases (asociación, herencia, dependencia, agregación, composición).

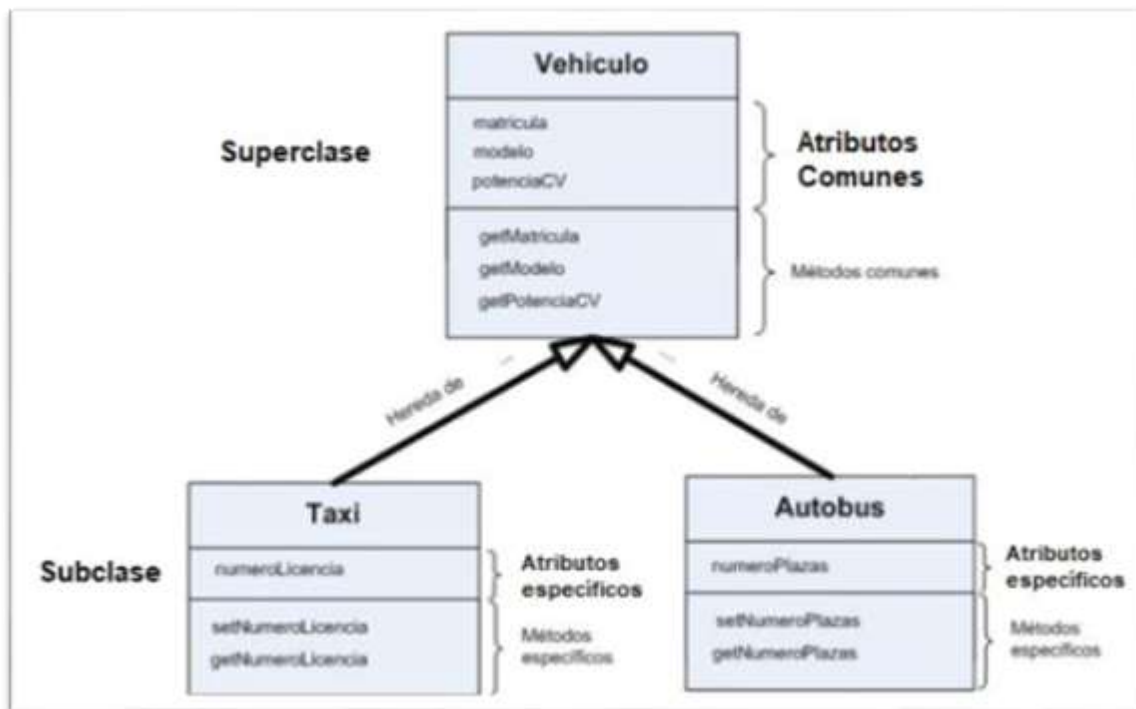
Relaciones comunes y su significado:

- **Asociación:** Comunicación entre clases (línea simple).
- **Herencia:** Una clase deriva de otra (flecha con triángulo).
- **Dependencia:** Uso temporal de una clase por otra (línea punteada).
- **Agregación:** Contención débil (rombo blanco).
- **Composición:** Contención fuerte (rombo negro).

Los diagramas ayudan a detectar errores de diseño y mejorar la comunicación en equipos de desarrollo.

3. Relación entre Diagramas de Clases y la POO

El diagrama de clases y la POO tienen una relación complementaria y secuencial: el diagrama actúa como plano (fase de análisis y diseño) y la POO como construcción (fase de implementación).



Cómo se complementan:

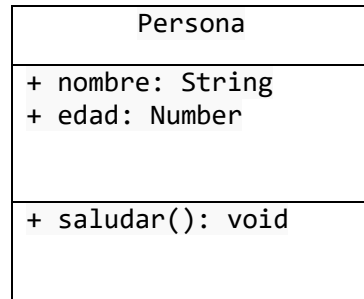
- El diagrama identifica clases, responsabilidades y relaciones.
- La POO transforma esas clases y relaciones en código (clases, herencia, interfaces, etc.).
- El diagrama facilita la detección temprana de problemas lógicos (reducción de retrabajo).
- Documenta decisiones de diseño para futuros mantenimientos.

Ventajas de usar ambos:

- Mejora la colaboración entre desarrolladores y analistas.
- Reduce la probabilidad de errores en la implementación.
- Facilita el mantenimiento, pruebas y escalabilidad del sistema.
- Sirve como material de capacitación y referencia técnica.

4. Ejemplo

Diagrama de clase (simplificado):



Implementado en POO

```
class Persona {  
  constructor(nombre, edad) {  
    this.nombre = nombre;  
    this.edad = edad;  
  }  
  
  saludar() {  
    console.log(`Hola, mi nombre es ${this.nombre} y tengo ${this.edad}  
años.`);  
  }  
}  
const ana = new Persona('Ana', 20);  
ana.saludar();
```

Ejemplo ampliado con herencia:

```
class Animal {
  constructor(nombre) {
    this.nombre = nombre;
  }
  sonido() {
    console.log('Sonido genérico');
  }
}

class Perro extends Animal {
  sonido() {
    console.log('Guau Guau!');
  }
}

const firulais = new Perro('Firulais');

firulais.sonido();
```

Este flujo (diagrama → clases → objetos) ilustra cómo el diseño guía la codificación y cómo POO materializa el modelo.

5. Conclusión

Los diagramas de clases y la POO son herramientas inseparables en el proceso de desarrollo de software bien diseñado. El diagrama de clases facilita la **visualización y planificación** del sistema, mientras que la POO permite **materializar** esa planificación en código modular y reutilizable. Su uso conjunto mejora la calidad del software, la comunicación entre equipos y la facilidad de mantenimiento.

6. Bibliografía

- ¿Qué es UML? (organización oficial) — <https://www.uml.org>
- Guía visual de diagrama de clases (tutorial práctico)
— <https://www.lucidchart.com/pages/es/diagrama-de-clases>
- POO en JavaScript — explicación y ejemplos (MDN)
— https://developer.mozilla.org/es/docs/Learn/JavaScript/Objects/Object-oriented_JS
- Ejemplos prácticos y ejercicios (Programiz)
— <https://www.programiz.com/javascript/object>
- Más diagramas UML (ejemplos y símbolos) — <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-class-diagram/>